

PHASMDAG-Core: Closure-Certified Finality for High-Throughput DAG Consensus

RobustDAG and the Separation of Ordering from Strong Finality
in Proof-of-Work BlockDAGs

Arthur Zhang

April 2026

Abstract

This paper sets out a preliminary formulation of *PHASMDAG-Core*, a conservative high-throughput DAG protocol that may also be described, at the level of design philosophy, as *RobustDAG*. The proposal is motivated by the contention that existing GhostDAG-family designs conflate three distinct objects: *ordering*, *dependency closure*, and *finality*. Recent work by Wu, Wei, Zhang, and Jiang (NDSS 2024) suggests that the *Decoupling Assumption*—a prerequisite for the security analyses of the major DAG-based PoW protocols—may fail under sufficiently high throughput, owing to Block Jam and Late-Predecessor effects. The present proposal therefore preserves a stable GhostDAG-style ordering core with fixed consensus parameters, but adds a closure-aware finality layer in which strong confirmation is granted only after deterministic, work-weighted closure-certificate conditions hold over a bounded support window. PHASMDAG-Core treats closure as a monotonic finality certificate rather than a volatile ordering weight. Immature closure may delay strong confirmation, but it must not by itself redirect canonical ordering. This conservative asymmetry is intended to bound the payoff of closure manipulation: adversarially delayed closure evidence may degrade latency, but should not become a cheap consensus-steering lever. Bi-objective closure-risk ordering is retained only as an experimental extension for future work, not as the core rule of the proposed protocol. The paper develops this v1 design sketch, states the principal theorem target as *certificate coherence* across honest nodes, and sets out a simulation and implementation roadmap rather than a completed security proof.

Keywords: DAG consensus; GhostDAG; closure-certified finality; late predecessor; block jam; semantic separation; finality semantics

1 Introduction

DAG-based Proof-of-Work consensus protocols—SPECTRE [4], PHANTOM/GhostDAG [5, 6], OHIE [9], and Prism [10]—represent the most promising direction for scaling Nakamoto consensus beyond the throughput ceiling imposed by chain-style protocols. By allowing blocks to reference multiple parents and thereby capture concurrency, these protocols achieve block rates that far exceed the network delay bound D .

However, Wu et al. [7] proved that this performance gain comes with a previously unaccounted security cost. Their *Congestible Blockchain Model* (CBM) shows that when block production rate f is high relative to network capacity, two phenomena emerge:

1. **Block Jam (BJ)**: when the total size of priority blocks in a time window exceeds the network’s relay capacity, these blocks cannot propagate within the assumed delay D .
2. **Late Predecessor (LP)**: a block’s propagation completion does not imply its acceptability—a block can only be accepted after *all* its predecessors have been received and validated. If a predecessor arrives later than its successor, the successor’s actual delay ($\Delta_{\text{actual}} = \Delta_{\text{prop}} + \Delta_{\text{proc}}$) exceeds its propagation delay alone.

Together, BJ and LP violate the *Decoupling Assumption* [7]: the assumption that priority blocks can propagate within fixed short delay D and be immediately accepted and built upon. All existing security proofs for DAG-based PoW protocols rely on this assumption.

Contribution summary.

1. **Root-cause diagnosis.** It is argued that the fundamental limitation of GhostDAG-family protocols lies not merely in the attackability of relay paths, but in the requirement that a single structural classifier should simultaneously represent canonical order, safe acceptability, and finality confidence. This conflation is largely invisible under optimistic network assumptions, but becomes a security liability under CBM.
2. **Protocol design.** A candidate protocol design is set out in which closure is introduced as a first-class *finality-certification* state, while the canonical ordering core is deliberately left structurally stable. PHASMDAG-Core preserves a GhostDAG-style blue-work lineage for tentative ordering, but grants strong confirmation only after a monotonic, work-weighted closure certificate has been issued.
3. **Preliminary security analysis.** A conservative security framing is proposed in which LP/BJ stress is intended to delay strong finality before it can silently weaken safety semantics. The central claim is not that closure manipulation is eliminated, but that its payoff is bounded by refusing to let immature closure directly steer canonical order.
4. **Formal modelling and verification roadmap.** A compositional formal model, a simulation-based evaluation methodology, and a phased verification programme are delineated around the theorem target of certificate coherence.

On the designation “PHASM”. The designation *PHASMDAG-Core* is intended to evoke *phase-separated semantics*: order, closure, and finality are treated as distinct protocol phases rather than being collapsed into a single structural score.

Claim status. This document is intentionally written as a protocol proposal rather than as a completed proof artifact. **Definitions** specify protocol intent. **Lemmas** and **Propositions** are preliminary and hold only under the modelling assumptions stated around them. **Conjectures** identify unresolved validation targets. The most important unresolved target is *certificate coherence*: whether raw closure observations and certificate-side thresholds converge sufficiently across honest nodes to support consistent strong confirmation.

Design principles.

1. **Immature closure delays strong confirmation; it does not directly penalise canonical order.**
2. **Closure is a monotonic certification layer, not a volatile ordering weight.**
3. **The v1 protocol is conservative:** fixed K , fixed support window H , no dynamic λ , no dynamic K , no closure bounty, and no telemetry-driven local consensus parameters.

The guiding intuition is simple: *a robust high-throughput DAG should parallelise ordering, not finality. Strong finality is serialized through dependency closure.*

2 Related Work and Theoretical Context

2.1 Throughput, Security, and Delay in PoW Consensus

The tension between throughput and network delay predates DAG-based consensus. Sompolinsky and Zohar’s high-rate Bitcoin analysis [1] already emphasised that short inter-block intervals and bounded delay cannot be treated independently. Formal longest-chain analyses subsequently made this dependence explicit through common-prefix, chain-growth, and chain-quality style guarantees under bounded-delay assumptions [2]. Zhang and Preneel [8] later unified these perspectives into a common-metrics framework, making it possible to compare protocol families under aligned security notions rather than under protocol-specific metrics.

Two considerations may be taken from that literature. First, performance claims in PoW systems are meaningful only relative to an explicit delay model. Second, security arguments become fragile when propagation, acceptance, and mining eligibility are silently conflated.

2.2 DAG-Based PoW Ordering Protocols

The DAG-based PoW literature can be organised by the manner in which concurrency is converted into order:

SPECTRE [4] provides fast confirmation through pairwise voting, but does not aim to maintain a global total order.

PHANTOM [5] formulates ordering as a k -cluster identification problem, thereby making explicit the relation between concurrency and anticone structure.

GhostDAG [6] replaces exact clustering with a greedy K -cluster approximation, yielding a practical structural order rule based on blue-set growth and blue-work accumulation.

OHIE [9] separates transaction handling from structural ordering through a numbered-chain construction.

Prism [10] decomposes consensus into proposal, voting, and committee roles, thereby achieving high throughput through role separation rather than through a single ordering object.

The present paper differs from these systems in one principal respect. The distinctive feature is not the use of a DAG per se, but the explicit separation of *order*, *closure maturity*, and *finality* as distinct protocol-level semantic objects.

2.3 Congestion, Delayed Acceptability, and Strategic Deviation

Wu et al. [7] identify the strongest current challenge to the security analysis of high-rate DAG protocols. Their Congestible Blockchain Model shows that propagation completion need not coincide with acceptability once block jam and late-predecessor effects are present. This result is directly relevant here because it shifts the analytical focus from pure relay delay to dependency visibility and processing delay.

The incentive literature is also relevant. Strategic withholding attacks were first articulated clearly in the selfish mining literature [3], where delayed publication changes the effective security margin even without violating PoW validity. Closure manipulation in PHASMDAG-Core is not identical to selfish mining, but it belongs to the same broad family of attacks in which timing asymmetry

and selective release alter the consensus-visible state. Any academically complete treatment of a closure-aware protocol must therefore address strategic publication incentives, not merely structural safety.

2.4 The Congestible Blockchain Model

Wu et al. [7] introduce the Congestible Blockchain Model (CBM), which relaxes the Uniform Delay Bounded Model (UDBM) used in prior security proofs.

Definition 2.1 (CBM, informal). Under CBM, each block B experiences a block-specific propagation delay upper bound D_B that may exceed the nominal network delay D . Specifically, D_B depends on:

1. The bandwidth utilisation at the time of B 's propagation (Block Jam).
2. The arrival status of B 's predecessors at each receiving node (Late Predecessor).

The actual delay for block B at node v is:

$$\Delta_{\text{actual}}(B, v) = \Delta_{\text{prop}}(B, v) + \Delta_{\text{proc}}(B, v) \quad (1)$$

where $\Delta_{\text{proc}}(B, v)$ is the processing delay caused by late predecessors of B that have not yet arrived at v .

Theorem 2.2 (Security degradation under CBM [7]). *For a DAG-based PoW protocol under LP attack with s node groups, the chain growth rate of affected blocks is:*

$$g = (1 - \sigma) \gamma f, \quad \gamma = \frac{\alpha}{1 + \alpha f \mathbb{E}[\Delta]} \quad (2)$$

where σ is the attacker's hash rate fraction, $\alpha = 1 - \sigma$ is the honest fraction, and $\mathbb{E}[\Delta]$ is the expected actual delay. The security threshold $\beta < \gamma$ shrinks as $\mathbb{E}[\Delta]$ increases.

2.5 The Decoupling Assumption

Definition 2.3 (Decoupling Assumption [7]). A DAG-based PoW protocol satisfies the Decoupling Assumption if there exists a class of priority blocks such that:

1. Priority blocks propagate to all honest nodes within fixed delay D .
2. Upon receipt, honest nodes can immediately accept priority blocks and build upon them.

Block Jam negates condition (1); Late Predecessor negates condition (2). Zhang and Preneel [8] further show that this assumption is not merely a technical convenience—it is *necessary* for the existing proof structure of GhostDAG-family protocols, because the K -parameter derivation fundamentally requires that the anticone of any block is bounded by K with high probability, which in turn requires that blocks are accepted within D .

3 Problem Statement: The Semantic Conflation

3.1 Three Conflated Semantics

The following conflation appears in GhostDAG-family protocols:

Definition 3.1 (Semantic Conflation). GhostDAG-family protocols use a single structural object—the blue score / K -cluster colouring—to simultaneously represent three distinct semantic roles:

1. **Order**: the canonical position of a block in the DAG's preferred history.

2. **Acceptability**: whether a block’s dependency world is sufficiently complete for honest miners to safely build upon it.

3. **Finality**: the economic confidence that a block will not be reorged.

Under UDBM with low f , these three roles approximately coincide. Under CBM with high f , they diverge: a block can be structurally well-ordered (high blue score) yet have an immature dependency world (low acceptability), and acceptability does not guarantee finality.

This conflation is not merely an implementation oversight; it is a *mathematical* property of protocols whose consensus functional is purely structural:

$$\text{GhostDAG: } \max_S U_{\text{order}}(S) \quad (\text{structural concurrency only}) \quad (3)$$

When $U_{\text{order}}(S)$ is the sole objective, there is no mechanism within the protocol to distinguish between “structurally legal but closure-fragile” support and “structurally legal and closure-mature” support. The weakness of GhostDAG under CBM is therefore not merely that the inherited K parameter may become poorly calibrated. The deeper difficulty is that structural ordering is tacitly treated as a proxy for dependency acceptability and finality. PHASMDAG-Core does not attempt to repair this by making ordering ever more adaptive. It instead refuses to let structural order stand in for strong finality.

3.2 Why Non-Consensus Mitigations Are Necessary But Insufficient

Several implementation-level mitigations are natural responses to the LP/BJ challenge:

- **Header-First Propagation**: decouple header acceptance from body completion, allowing GhostDAG to progress on header closure alone.
- **Dependency-Aware Relay Priority**: prioritise relay of blocks that unblock the most pending successors.
- **Telemetry**: expose dependency-completion and orphan-age metrics.
- **Adaptive Safety Margins**: locally increase conservatism when network stress is detected.

These mitigations are valuable and should be pursued whenever possible. However, they share a common limitation: they protect a protocol whose core mathematical object remains structurally blind to closure.

Proposition 3.2. *Engineering mitigations that preserve the purely structural consensus objective $\max U_{\text{order}}(S)$ can reduce the magnitude of closure fragility but cannot detect it, certify it, or let it gate strong finality within the consensus protocol itself. Therefore, the security guarantee remains dependent on the Decoupling Assumption, albeit at a lower effective delay.*

Header-first, for example, narrows the LP attack surface from the body layer to the header layer, but does not eliminate it. The question “is this block’s dependency world mature enough for strong finality?” remains unanswerable within the protocol—it is answered only by heuristics in the implementation layer. This semantic mismatch is summarised visually in Figure 1.

3.3 Formal Problem Definition

Definition 3.3 (The PHASMDAG-Core Problem). Design a DAG consensus protocol that:

1. preserves GhostDAG-style high-throughput structural ordering,
2. exposes dependency closure as a protocol-visible finality condition,
3. prevents closure evidence from becoming a cheap ordering-manipulation vector,

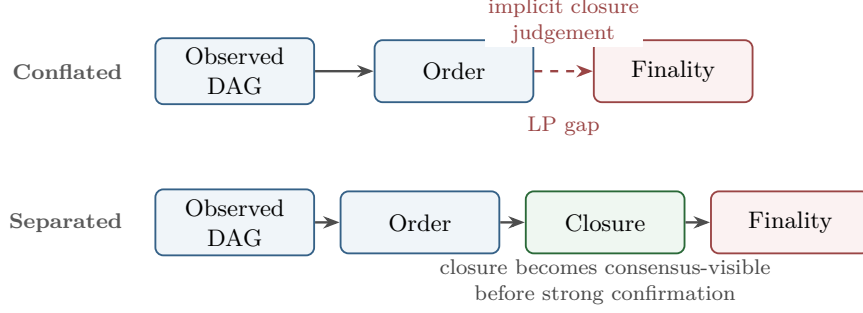


Figure 1: Why an explicit closure layer is needed. Under LP attack, structural ordering can progress before the dependency world is mature enough for strong confirmation. PHASMDAG-Core inserts closure as an explicit protocol stage rather than leaving that judgement to implementation heuristics.

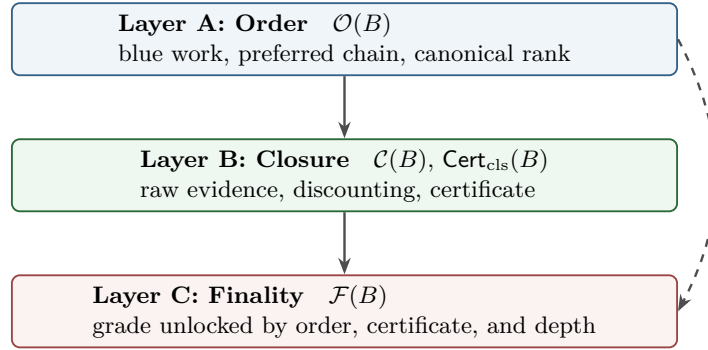


Figure 2: The three semantic layers of PHASMDAG-Core. The Order layer feeds the Closure layer; both jointly determine the Finality layer. The dashed arrow indicates that Order also feeds Finality directly, but strong confirmation is not unlocked until the Closure layer issues a certificate.

4. grants strong confirmation only after closure certification, and
5. degrades under LP/BJ by delaying strong finality rather than silently weakening safety semantics.

4 Protocol Design

4.1 Three Semantic Layers

PHASMDAG-Core reasons over three explicit semantic layers:

Layer A — Order Layer determines structural support and preferred ordering relations in the DAG. In PHASMDAG-Core- v_1 , this layer is fixed to a GhostDAG-style *blue-work lineage*; alternative order cores are future variants, not part of the base protocol.

Layer B — Closure Layer measures whether a block and its dependency world are sufficiently mature, stable, and coherently visible across honest nodes, and promotes only sufficiently stable evidence to a monotonic certificate. This is the additional layer absent from GhostDAG-family protocols.

Layer C — Finality Layer derives confirmation/finality from both order and certified closure, rather than from order alone.

The relationship between layers is illustrated in Figure 2.

4.2 Core Quantities

For each block B in the observed DAG $\mathcal{G} = (V, E)$, PHASMDAG-Core tracks four objects: the structural order score $\mathcal{O}(B)$, the raw closure observation $\mathcal{C}(B)$, the monotonic closure certificate $\text{Cert}_{\text{cls}}(B)$, and the resulting finality grade $\mathcal{F}(B)$.

Definition 4.1 (Order Strength). For PHASMDAG-Core- v_1 , the *order strength* of block B is the block's position on the GhostDAG blue-work lineage:

$$\mathcal{O}(B) := \text{BlueWork}(B), \quad \mathcal{O}(B) \in \mathbb{R}_{\geq 0} \quad (4)$$

a structural support quantity measuring how strongly the DAG structure supports placing B in the canonical history. The canonical order rank $r(B)$ is the deterministic rank induced by this blue-work order core.

Definition 4.2 (Relevant Support Window). Let $H \in \mathbb{N}$ be a protocol parameter. The *relevant support window* of block B is

$$\mathcal{W}_H(B) := \{D \in \text{desc}(B) : 0 < r(D) - r(B) \leq H\}. \quad (5)$$

All closure metrics in PHASMDAG-Core- v_1 are computed over $\mathcal{W}_H(B)$ so that they are bounded, deterministic, and auditable.

Definition 4.3 (Closure Evidence Sub-Metrics). Let $W(B)$ denote the PoW work attributed to block B . For each $D \in \mathcal{W}_H(B)$, let $\mathbf{1}_{\text{closed}}(D) = 1$ if all protocol-required predecessor objects of D have been observed and validated at the evaluating node, and 0 otherwise. Let $c^*(B)$ denote the canonical child of B on the blue-work lineage, and let $\pi_B(D)$ denote the first child of B on the path from B to D . Let $\text{Front}_H(B)$ be the set of maximal blocks in $\mathcal{W}_H(B)$, and let $f^*(B) \in \text{Front}_H(B)$ be the frontier tip that extends $c^*(B)$ when such a tip exists. The sub-metrics are defined by:

$$\mathcal{C}_{\text{dep}}(B) = \frac{\sum_{D \in \mathcal{W}_H(B)} W(D) \mathbf{1}_{\text{closed}}(D)}{\sum_{D \in \mathcal{W}_H(B)} W(D)} \quad (6)$$

$$\mathcal{C}_{\text{coh}}(B) = \frac{\sum_{D \in \mathcal{W}_H(B)} W(D) \mathbf{1}[\pi_B(D) = c^*(B)]}{\sum_{D \in \mathcal{W}_H(B)} W(D)} \quad (7)$$

$$\mathcal{C}_{\text{fr}}(B) = \frac{W(f^*(B))}{\sum_{F \in \text{Front}_H(B)} W(F)} \quad (8)$$

with the convention that each ratio is 0 when its denominator is 0. Define also the support-window work evidence

$$\text{Work}_H(B) := \sum_{D \in \mathcal{W}_H(B)} W(D). \quad (9)$$

Definition 4.4 (Raw Closure Observation). The *raw closure observation* of block B is the deterministic aggregation of the three closure sub-metrics:

$$\mathcal{C}(B) = g_{v_1}(\mathcal{C}_{\text{dep}}(B), \mathcal{C}_{\text{coh}}(B), \mathcal{C}_{\text{fr}}(B)) \quad (10)$$

where PHASMDAG-Core- v_1 uses the weighted geometric mean

$$g_{v_1}(x, y, z) = x^{\eta_{\text{dep}}} y^{\eta_{\text{coh}}} z^{\eta_{\text{fr}}}, \quad \eta_{\text{dep}} + \eta_{\text{coh}} + \eta_{\text{fr}} = 1. \quad (11)$$

This aggregation makes a low value in any one dimension visibly reduce the overall raw closure score. For typographic continuity, the paper writes $\mathcal{C}(B)$ for this node-local quantity. It is intentionally treated as an intermediate observation rather than as a direct trigger for strong confirmation.

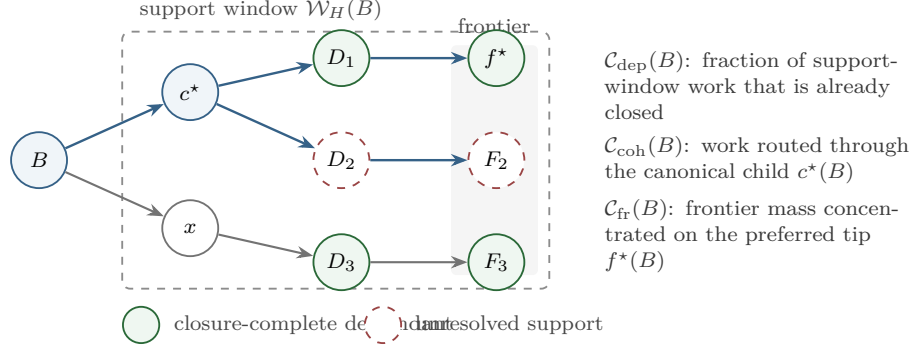


Figure 3: Closure decomposition around block B . The support window $\mathcal{W}_H(B)$ supplies the auditable evidence set; $\mathcal{C}_{\text{dep}}(B)$ measures how much of that work is closure-complete, $\mathcal{C}_{\text{coh}}(B)$ measures how much support flows through the canonical child $c^*(B)$, and $\mathcal{C}_{\text{fr}}(B)$ measures whether frontier work is concentrated on the preferred tip $f^*(B)$. In PHASMDAG-Core- v_1 , the first two quantities are mandatory Grade 2 inputs, while $\mathcal{C}_{\text{fr}}(B)$ is reserved for diagnostics or deep settlement.

Definition 4.5 (Closure Certificate). For PHASMDAG-Core- v_1 , the certificate-side closure components are

$$\mathcal{C}_{\text{dep}}^{\text{cert}}(B) = \frac{\sum_{D \in \mathcal{W}_H(B)} W(D) \mathbf{1}_{\text{closed}}(D)}{\text{Work}_H(B)} \quad (12)$$

$$\mathcal{C}_{\text{coh}}^{\text{cert}}(B) = \frac{\sum_{D \in \mathcal{W}_H(B)} W(D) \mathbf{1}[\pi_B(D) = c^*(B)]}{\text{Work}_H(B)}. \quad (13)$$

Define the certificate predicate

$$\text{ValidCert}_C(B) := [\mathcal{C}_{\text{dep}}^{\text{cert}}(B) \geq \theta_{\text{dep}}] \wedge [\mathcal{C}_{\text{coh}}^{\text{cert}}(B) \geq \theta_{\text{coh}}] \wedge [\text{Work}_H(B) \geq \theta_{\text{work}}]. \quad (14)$$

Block B is said to be *closure-certified*, written $\text{Cert}_{\text{cls}}(B) = 1$, iff $\text{ValidCert}_C(B)$ holds; once issued, the certificate is monotonic.

Remark 4.6. $\mathcal{C}(B)$ is *not* a function of observed propagation delay. It measures structural properties of the dependency world (completeness, stability, coherence), not temporal observations. This is a deliberate design choice: observed delays are node-specific and vulnerable to topology bias [7], whereas structural completeness can, in principle, be determined deterministically from the DAG structure that honest nodes eventually agree on. Strong confirmation is therefore tied not to the live score $\mathcal{C}(B)$ itself, but to the monotonic event $\text{Cert}_{\text{cls}}(B) = 1$.

Remark 4.7 (Why $\mathcal{C}_{\text{fr}}(B)$ is demoted in v_1). Equation (6) and Equation (7) are retained as the mandatory Grade 2 certificate inputs because they speak directly to dependency closure and coherent descendant support. By contrast, $\mathcal{C}_{\text{fr}}(B)$ is more exposed to frontier steering and local tip instability. PHASMDAG-Core therefore keeps $\mathcal{C}_{\text{fr}}(B)$ as an advisory or deep-settlement quantity in v_1 , rather than making it a brittle Grade 2 gate.

Figure 3 shows how these three quantities are read off from the local descendant structure around a block.

4.3 Closure-Certified Finality, Not Closure-Weighted Ordering

Definition 4.8 (Order-Preservation Principle). Canonical order in PHASMDAG-Core- v_1 is determined by the GhostDAG-style blue-work order core alone. Immature closure may delay strong confirmation, but it must not by itself redirect canonical ordering.

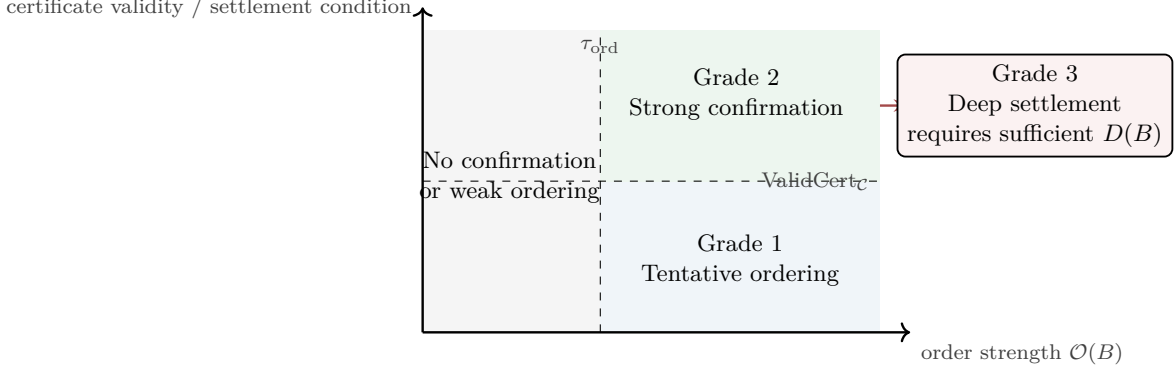


Figure 4: Schematic confirmation-grade geometry. A robust high-throughput DAG should parallelise ordering, not finality. Grade 2 is unlocked by certificate validity rather than by a volatile local closure score; Grade 3 adds deep settlement on top of Grade 2.

Definition 4.9 (Certificate-Gated Finality). Let $D(B)$ denote the settlement-depth or descendant-work condition used for deep settlement. The protocol exposes three grades:

$$\text{Grade}_1(B) \iff \mathcal{O}(B) \geq \tau_{\text{ord}}, \quad (15)$$

$$\text{Grade}_2(B) \iff \text{Grade}_1(B) \wedge \text{ValidCert}_C(B), \quad (16)$$

$$\text{Grade}_3(B) \iff \text{Grade}_2(B) \wedge D(B) \geq \tau_{\text{settle}} \wedge [\mathcal{C}_{\text{fr}}(B) \geq \theta_{\text{fr}}^{\text{deep}}]_{\text{optional}}. \quad (17)$$

The finality grade $\mathcal{F}(B)$ is the greatest $i \in \{0, 1, 2, 3\}$ such that $\text{Grade}_i(B)$ holds, with Grade_0 denoting the absence of protocol-level confirmation.

PHASMDAG-Core treats closure as a monotonic finality certificate rather than a volatile ordering weight. Strong finality is serialized through dependency closure. This is the protocol’s central asymmetry: positive closure evidence unlocks stronger confirmation, whereas missing closure evidence does not directly punish canonical order.

Grade 1 — Tentative Ordering The block is structurally well placed in the GhostDAG-style order core.

Grade 2 — Closure-Certified Strong Confirmation The block satisfies the order threshold and has a valid closure certificate. Raw closure observations alone are insufficient.

Grade 3 — Deep Settlement The block is already Grade 2 and additionally satisfies a deep-settlement condition. Frontier stability may enter here as an optional strengthening, but it is not required for Grade 2 in v_1 .

This replaces the traditional “one confirmation number fits all” model with semantically differentiated confirmation objects. The geometry of these grades is illustrated in Figure 4.

4.4 Diagnostic Closure-Risk Metrics and Optional Extensions

Definition 4.10 (Diagnostic Closure-Risk Functional). For a candidate set S , define the diagnostic closure-fragility metric

$$R_{\text{closure}}(S) = a \sum_{B \in S} (1 - \mathcal{C}_{\text{dep}}(B)) + b \sum_{B \in S} (1 - \mathcal{C}_{\text{coh}}(B)) + c \sum_{B \in S} (1 - \mathcal{C}_{\text{fr}}(B)) + d \cdot \text{FragilityCoupling}(S) \quad (18)$$

where $a, b, c, d \geq 0$ and

$$\mathcal{U}(B) := \{D \in \mathcal{W}_H(B) : \mathbf{1}_{\text{closed}}(D) = 0\}, \quad (19)$$

$$\text{FragilityCoupling}(S) := \frac{1}{|P_2(S)|} \sum_{\{B_i, B_j\} \in P_2(S)} \frac{\sum_{D \in \mathcal{U}(B_i) \cap \mathcal{U}(B_j)} W(D)}{\sum_{D \in \mathcal{U}(B_i) \cup \mathcal{U}(B_j)} W(D)}, \quad (20)$$

with value 0 when the denominator is 0.

Remark 4.11 (Status of R_{closure} and λ in v_1). In PHASMDAG-Core, R_{closure} is a diagnostic and simulation object in v_1 , not a hard canonical-ordering penalty. It may still be useful for empirical comparison, reporting closure fragility, or studying optional miner-local template preferences in simulations. However, if R_{closure} directly affects canonical ordering, closure manipulation becomes a consensus-steering problem. PHASMDAG-Core therefore deliberately avoids hard R_{closure} penalties in the deployed ordering rule.

Remark 4.12 (λ is not consensus-critical in v_1). PHASMDAG-Core has no dynamic λ , no telemetry-driven local λ , and no per-node adaptive λ in the consensus rule. If a future extension studies a λ -bearing objective, it should be understood as a separate protocol family that requires its own incentive-compatibility analysis.

4.5 Confirmation Invariants

Invariant 4.13 (Tentative Ordering Bounded Churn). There exists a protocol parameter ρ_{tent} such that once a block B attains Grade 1, any later honest view that still places B in the canonical order may change its rank by at most ρ_{tent} . Tentative ordering may move, but only within a bounded local reorder depth.

Invariant 4.14 (Strong Confirmation Agreement). No two honest nodes may assign Grade 2 to conflicting blocks at the same virtual position in the canonical order. Strong confirmation is therefore a protocol object, not merely a UI label.

Invariant 4.15 (Deep Settlement Tail). There exists a protocol target $\varepsilon_{\text{settle}}$ such that any block assigned Grade 3 has rollback probability at most $\varepsilon_{\text{settle}}$ under the stated network and adversarial assumptions.

5 Security Analysis under CBM

5.1 Threat Model

The CBM threat model of [7] is adopted with three layers:

1. **Analytical layer:** an abstract CBM attacker who can partition honest nodes into s groups and selectively delay block delivery.
2. **Realisable layer:** attacks implementable via BGP hijacking, Eclipse attacks, or strategic peer selection.
3. **Implementation layer:** representative node behaviour under stress, including orphan-pool dynamics and header-body skew.

5.2 Closure Manipulation and Payoff Bounding

This section is intentionally conservative. It does *not* claim that PHASMDAG-Core eliminates closure manipulation. Rather, the design objective is to limit the payoff of such manipulation by refusing to let immature closure directly steer canonical order.

Proposition 5.1 (Payoff-bounding design intention). *Under the PHASMDAG-Core- v_1 design, an attacker may delay closure evidence and thereby postpone Grade 2 confirmation. However, because closure does not directly alter canonical order in v_1 , such manipulation is primarily a latency-griefing vector rather than an ordering-redirection vector.*

Proof sketch. By Definition 4.8, canonical order depends only on the GhostDAG-style order core and its fixed consensus parameters. Closure evidence enters at the Grade 2 boundary through Equations (14) and (16), not through the ordering rule itself. Withholding or selectively releasing predecessor information may therefore delay the moment at which $\text{ValidCert}_C(B)$ becomes true, but it does not by itself re-rank a block that is already canonically ordered. $\square$

The protocol deliberately bounds the payoff of closure manipulation: adversarially delayed closure evidence can degrade latency, but should not become a cheap consensus-steering lever.

Predecessor withholding. An attacker may withhold a predecessor block B^* so as to delay dependency closure for descendants that rely upon it. In GhostDAG, such delay risks being mistaken for weak structural support. In PHASMDAG-Core, the intended effect is narrower: the victim’s block may remain in Grade 1 for longer, but tentative canonical order is not thereby revoked.

Timed closure release. An attacker may release previously withheld closure evidence at a moment chosen to accelerate a preferred branch’s transition from Grade 1 to Grade 2. This remains a first-class strategic threat. The v_1 response is not to claim that the threat disappears, but to ensure that its effect is confined chiefly to certification latency rather than to canonical-order steering.

Frontier steering. Because frontier state can be noisy and locally unstable, the v_1 protocol does not let $C_{\text{fr}}(B)$ serve as a mandatory Grade 2 certificate gate. This reduces the extent to which local frontier manipulation can become a brittle strong-confirmation trigger.

R_{closure} gaming. If R_{closure} were made consensus-critical, miners could attempt to shape closure fragility metrics so as to steer order selection itself. PHASMDAG-Core therefore excludes R_{closure} from the v_1 canonical-order rule and retains it only as a diagnostic or simulation object.

These attacks cannot presently be eliminated. The design objective is instead to prevent them from maturing into safety failures or canonical-order steering attacks. That is the principal sense in which PHASMDAG-Core claims robustness under CBM pressure.

5.3 Comparison with GhostDAG under LP Attack

Table 1 highlights the central asymmetry: PHASMDAG-Core does not claim to eliminate LP/BJ stress. It instead aims to make the semantic consequences of such stress explicit by withholding strong finality until closure has been certified.

5.4 Certificate Coherence Theorem Target

Under LP attack, different honest nodes may still see different local evidence for a period of time. The central theorem target is therefore no longer merely closure convergence, but certificate coherence across honest nodes.

Table 1: Preliminary security behaviour comparison under LP attack

Property	GhostDAG	PHASMDAG-Core
Tentative-order basis	structural order also carries confirmation semantics	same GhostDAG-style order core
Strong-finality basis	structural order is used as proxy	$\mathcal{O}(B) \wedge \text{ValidCert}_{\mathcal{C}}(B)$
Effect of predecessor withholding	may silently degrade both order and confirmation semantics	primarily delays Grade 2 certification
Effect of frontier steering	entangled with confirmation semantics	mostly pushed to Grade 3 or diagnostics
Consensus-steering payoff from missing closure	implicit and potentially large	deliberately bounded by design

Conjecture 5.2 (Certificate coherence theorem target). *Under CBM with honest majority, if some honest node v issues $\text{Cert}_{\text{cls}_v}(B) = 1$, then within bounded additional time $\Delta_{\text{coherence}}$ all honest nodes will either also issue $\text{Cert}_{\text{cls}}(B) = 1$ or reject certification under the same deterministic support-window evidence rules.*

If established, Definition 5.2 would show that the Grade 2 predicate is not merely a local heuristic but a consensus-meaningful object. Without such certificate coherence, PHASMDAG-Core would remain only a descriptively attractive semantics rather than a defensible finality rule.

5.5 Heuristic Note on Confirmation Growth

Any quantitative dominance claim should be treated as heuristic rather than as the primary security result of the paper. The primary safety improvement in PHASMDAG-Core is semantic: strong confirmation is withheld until dependency closure is certified.

Remark 5.3 (Optional analytical intuition). It remains heuristically useful to distinguish tentative-order growth from Grade 2 confirmation growth. Tentative order may continue to inherit the GhostDAG-style expression $g_{\text{PHASM}}^{(1)} = (1 - \sigma)\gamma f$, while Grade 2 growth may be viewed as scaled by a certificate-issuance factor $p_{\text{cert}}(T)$. However, this intuition is not the principal safety claim of the protocol and should be understood as an analytical convenience only.

6 Formal Modelling Approach

This section develops a compositional formal model aligned with the three semantic layers.

6.1 Compositional State Machine

Each block B maintains three logical state components:

1. **Structural state** $\Sigma_{\text{ord}}(B)$: parent set, selected-parent relation, blue-work score, ordering rank. This is essentially the existing GhostDAG state.
2. **Closure state** $\Sigma_{\text{cls}}(B)$: relevant support window $\mathcal{W}_H(B)$, the three sub-metrics $\mathcal{C}_{\text{dep}}(B)$, $\mathcal{C}_{\text{coh}}(B)$, $\mathcal{C}_{\text{fr}}(B)$, unresolved-support set $\mathcal{U}(B)$, support-window work evidence $\text{Work}_H(B)$, certificate-side metrics $\mathcal{C}_{\text{dep}}^{\text{cert}}(B)$ and $\mathcal{C}_{\text{coh}}^{\text{cert}}(B)$, raw closure observation $\mathcal{C}(B)$, and certificate status $\text{Cert}_{\text{cls}}(B)$.

3. **Finality state** $\Sigma_{\text{fin}}(B)$: tentative-order status, closure-certified strong-confirmation status, deep settlement status, and finality grade $\mathcal{F}(B)$.

The full state machine is the product $\Sigma(B) = \Sigma_{\text{ord}}(B) \times \Sigma_{\text{cls}}(B) \times \Sigma_{\text{fin}}(B)$, with admissible transitions defined by the protocol rules. The transition order is strict: Σ_{ord} is computed first (since closure depends on structural state), then Σ_{cls} , then Σ_{fin} .

6.2 Local Views and Disagreement Metrics

The key modelling difficulty is that PHASMDAG-Core is evaluated under partial and heterogeneous visibility. The protocol therefore requires an explicit notation for node-local views and for cross-node disagreement in closure scores.

Definition 6.1 (Node-Local DAG View). Let $\mathcal{G}_v(t)$ denote the accepted header-DAG observed by honest node v at time t . All protocol quantities are node-local during execution:

$$\mathcal{O}_v(B, t), \quad \mathcal{C}_v(B, t), \quad \text{Cert}_{\text{cls}_v}(B, t), \quad \mathcal{F}_v(B, t).$$

Consensus safety requires that these local quantities converge sufficiently fast after network stabilisation.

Definition 6.2 (Closure Disagreement). For any block B visible to the honest node set H_t at time t , define the honest-node closure disagreement by

$$\Delta_{\mathcal{C}}(B, t) := \max_{u, v \in H_t} |\mathcal{C}_u(B, t) - \mathcal{C}_v(B, t)|. \quad (21)$$

The central modelling objective is to show that $\Delta_{\mathcal{C}}(B, t)$ becomes small quickly enough to support consistent Grade 2 decisions.

Definition 6.3 (Certificate-Coherent Strong Confirmation Target). A block B is said to satisfy the *certificate-coherent strong-confirmation target* at time t if

$$\forall v \in H_t, \text{Cert}_{\text{cls}_v}(B, t) = 1 \quad \text{and} \quad \Delta_{\mathcal{C}}(B, t) \leq \varepsilon_{\text{coh}} \quad (22)$$

for a protocol tolerance $\varepsilon_{\text{coh}} > 0$. Definition 5.2 may then be read as a theorem target asserting eventual attainment of (22) for structurally supported honest blocks.

This convergence target is visualised in Figure 5.

6.3 Block Processing Algorithm

Algorithm 1 specifies the high-level block processing pipeline for PHASMDAG-Core, making the three-layer computation order explicit.

6.4 Raw Closure Observation and Certificate Computation

The most critical modelling challenge is the computation of raw closure observations together with the monotonic closure certificate. PHASMDAG-Core treats this as a deliberately conservative design space: one deployable certificate-oriented instantiation is standardised for v_1 , while more aggressive closure-aware ordering ideas are kept outside the core protocol.

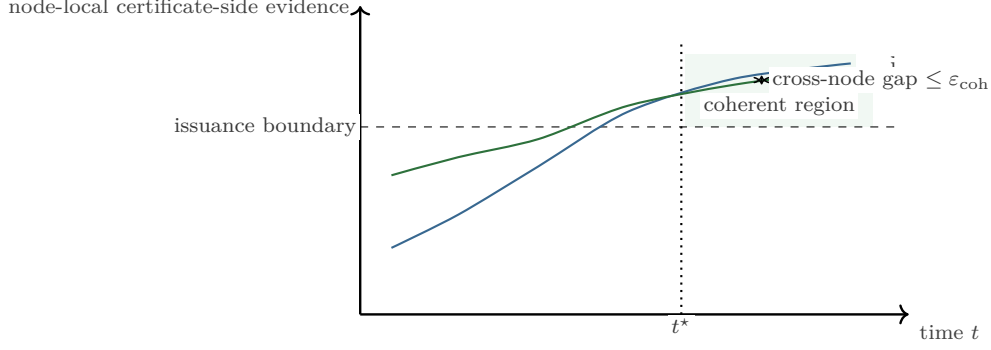


Figure 5: Closure coherence as a convergence target. Strong confirmation should only become globally meaningful once node-local certificate-side evidence lies near the issuance boundary and cross-node disagreement is sufficiently small.

Approach A: Deterministic closure certification over a bounded support window (v_1).

Approach A is the **only** standardised closure computation in PHASMDAG-Core- v_1 . Its simplest descendant-attestation form is the PoW-weighted statistic

$$\mathcal{C}_A(B) = \frac{\sum_{B' \in \text{desc}(B), \mathbf{1}_{\text{closed}}(B')=1} W(B')}{\sum_{B' \in \text{desc}(B)} W(B')}. \quad (23)$$

Here $W(B')$ denotes the PoW weight, accumulated work, or difficulty-equivalent contribution of descendant block B' . This makes closure evidence an economic-security quantity rather than a raw descendant count, and prevents a minority miner from cheaply inflating closure with low-value descendants alone. It instantiates $\mathcal{C}_{\text{dep}}(B)$, $\mathcal{C}_{\text{coh}}(B)$, and $\mathcal{C}_{\text{fr}}(B)$ exactly as in Equations (6) to (8); only the first two enter the Grade 2 certificate rule in v_1 . Equation (23) is the unwindowed intuition; PHASMDAG-Core- v_1 implements the same idea through the bounded support-window version of Equations (6), (7) and (9). It uses only structural evidence observable in the accepted header-DAG plus the validation state required for $\mathbf{1}_{\text{closed}}$. Certificate issuance is then determined by Equation (14). No wall-clock propagation delay, dynamic parameter, or frontier-only trigger is required for Grade 2 in this conservative instantiation. This makes the construction deterministic, decomposable, auditable, and closer to a plausible deployable core.

Approach B: Experimental closure-weighted ordering extension. An alternative line of design would feed a closure-fragility functional directly back into ordering, for example through a score of the form

$$\Phi(S) = U_{\text{order}}(S) - \lambda R_{\text{closure}}(S). \quad (24)$$

Such an approach is attractive because it appears to internalise LP/BJ risk inside the ordering rule itself. However, it also turns missing closure evidence into a strategic resource that may be used to steer order. For that reason, PHASMDAG-Core excludes this idea from v_1 and treats it as future work.

Remark 6.4 (Version policy for closure computation). Approach A is the reference instantiation for PHASMDAG-Core- v_1 . Approach B remains explicitly experimental. This is a versioning decision, not merely a modelling preference: the base protocol optimises for semantic separation, determinism, and bounded manipulation payoff before attempting closure-aware ordering.

Table 2: Core vs. experimental closure-handling strategies

Property	Approach A	Approach B
Deterministic across nodes	Strong	Weak–Medium
Ordering stability	Strong	Lower
Formal proof friendliness	High	Low
Topology bias risk	Low	Higher
Consensus-steering risk	Lower	Higher

6.5 Parameter Space

The core protocol introduces the following parameters that must be analysed formally:

6.6 Formal Verification Targets

The following properties are targets for formal verification:

1. **Tentative bounded churn:** prove or bound Definition 4.13 under the blue-work order core.
2. **Strong-confirmation safety:** prove Definition 4.14, namely that no two honest nodes assign conflicting Grade 2 confirmations at the same virtual position.
3. **Settlement tail:** prove or upper-bound Definition 4.15.
4. **Liveness:** every honestly mined block eventually reaches tentative ordering (Grade 1) if the network eventually stabilises.
5. **Certificate coherence:** establish or refute Definition 5.2, which is the central theorem target of the protocol.
6. **Payoff-bounding validation:** determine whether the bounded manipulation-payoff framing in Definition 5.1 survives a fuller CBM analysis or only a simulation-level approximation.

7 Engineering Verification

7.1 Simulation Methodology

The simulation programme is based on an extension of the SimBlock [11] framework to model CBM conditions:

1. **Network model:** replace the uniform delay model with CBM, incorporating bandwidth limits and per-block propagation delays.
2. **LP attack simulation:** implement the s -group partition attack from [7], allowing the attacker to selectively delay block delivery to specific node groups.
3. **Bandwidth congestion:** simulate Block Jam by enforcing a global bandwidth cap and measuring propagation delays as a function of utilisation.
4. **Closure tracking:** augment each simulated node with the v_1 raw-closure and certificate computation (Approach A), tracking both $\mathcal{C}(B)$ and the issuance event $\text{Cert}_{\text{cls}}(B) = 1$ over time.

7.2 Evaluation Metrics

The evaluation is organised around the following metrics, following the common-metrics framework of [8]:

Table 3: Protocol parameter space

Parameter	Type	Constraints
K	GhostDAG order parameter	Fixed consensus constant inherited from the GhostDAG-style ordering core
H	Closure window	Support-horizon size for $\mathcal{W}_H(B)$; must be large enough to capture relevant descendant evidence
$\eta_{\text{dep}}, \eta_{\text{coh}}, \eta_{\text{fr}}$	Aggregation	Non-negative and sum to 1 in Equation (11); diagnostic only
τ_{ord}	Order threshold	Minimum structural support for Grade 1
$\theta_{\text{dep}}, \theta_{\text{coh}}$	Certificate thresholds	Minimum dependency-completeness and coherent-support evidence for $\text{ValidCert}_{\mathcal{C}}(B)$
θ_{work}	Work threshold	Minimum support-window work needed before a certificate may be issued
τ_{settle}	Settlement threshold	Minimum deep-settlement condition for Grade 3
$\theta_{\text{fr}}^{\text{deep}}$	Optional frontier stability	May be used as a conservative Grade 3 strengthening, not as a Grade 2 gate
ε_{coh}	Coherence tolerance	Maximum admissible cross-node disagreement when Grade 2 becomes globally meaningful
ρ_{tent}	Tentative ordering	Maximum permitted rank churn for Grade 1 (Definition 4.13)
$\varepsilon_{\text{settle}}$	Settlement target	Maximum rollback tail for Grade 3 (Definition 4.15)
λ and R_{closure}	Experimental extension	Not consensus-critical in v_1 ; retained only for diagnostics, simulation, or future variants
$\mathcal{C}$ definition	Protocol design	v_1 : Approach A only; Approach B is experimental (Definition 6.4)

Table 4: Evaluation metrics

Metric	Description
Chain growth rate g	Blocks per second entering the canonical order
Certificate issuance rate $p_{\text{cert}}(T)$	Fraction of tentatively ordered blocks that become certified within horizon T
Realised TPS	Actual transaction throughput under imperfect networks
Tentative confirmation latency	Time from block creation to Grade 1
Strong confirmation latency	Time from block creation to Grade 2
Deep settlement latency	Time from block creation to Grade 3
Certificate coherence time	Time until honest nodes agree sufficiently for $\text{Cert}_{\text{cls}}(B) = 1$ to become globally meaningful
P95/P99 finality tail	Tail behaviour of confirmation latency under stress

7.3 Comparative Evaluation Plan

1. **Baseline:** GhostDAG under UDBM (optimistic reference).
2. **GhostDAG under CBM:** GhostDAG with LP attack and bandwidth congestion, no mitigations.
3. **GhostDAG + engineering mitigations:** GhostDAG with header-first, dependency-aware relay, and telemetry.
4. **PHASMDAG-Core under CBM:** PHASMDAG-Core with $\mathcal{C}$ computation (Approach A) and certificate issuance, under the same CBM conditions.

The comparison between configurations (3) and (4) directly tests Definition 3.2: does the protocol-level closure awareness of PHASMDAG-Core provide security benefits beyond what engineering mitigations can achieve?

7.4 Engineering Priorities for v1

The conservative presentation of PHASMDAG-Core is intended to make the protocol look buildable rather than merely suggestive. The minimal engineering package for any v_1 prototype should include:

1. **Header-first propagation**, so that ordering can continue even when full dependency availability lags.
2. **Dependency-aware relay**, so that blocks resolving the largest missing-predecessor pockets are propagated first.
3. **Orphan and missing-predecessor recovery**, so that unresolved support windows can be repaired promptly once absent predecessors are discovered.
4. **Compact closure-evidence caching**, so that support-window work and coherent-support counters are maintained incrementally rather than recomputed from scratch.
5. **A fixed support horizon H** , so that both analysis and implementation remain bounded.

7.5 Complexity and Incremental Computation

The introduction of Σ_{cls} and Σ_{fin} state machines, fixed support-window tracking, and closure-evidence caches raises legitimate concerns about computational overhead. A protocol that is theoretically sound but practically unrunnable serves no one. Incremental computation is therefore stipulated as a strict prerequisite for v_1 , not an afterthought.

The critical metrics that must be evaluated and bounded before a PHASMDAG-Core prototype is considered viable are:

1. **Amortised per-block update cost for raw closure and certificate state:** Computing $\mathcal{C}(B)$ for a new block requires updating the three sub-metrics over the affected support windows; issuing $\text{Cert}_{\text{cls}}(B)$ also requires updating certificate-side thresholds over the same local region. Under Approach A (Equations (6) to (8)), this can be formulated incrementally: when a new block B_{new} arrives, only its direct work contribution and its canonical-child / frontier relations need to be propagated to affected ancestors. The target is $O(|\text{parents}|)$ per block, not $O(|\text{ancestors}|)$.
2. **Memory footprint of closure-state indices and attestation caches:** Each block must store: (a) the cumulative work mass needed for $\mathcal{C}_{\text{dep}}(B)$, (b) support-window routing metadata needed for $\mathcal{C}_{\text{coh}}(B)$, and (c) frontier concentration metadata needed for $\mathcal{C}_{\text{fr}}(B)$, together with a compact certificate-evidence cache for $\text{Work}_H(B)$. These are all bounded by the finite support window H and can be encoded as compact additive counters plus small frontier indices rather than as a full descendant list.

Table 5: Phased verification roadmap

Phase	Timeline	Deliverables
1	0–6 mo	Formal model: define $\mathcal{O}$, $\mathcal{C}_{\text{dep}}$, $\mathcal{C}_{\text{coh}}$, $\mathcal{C}_{\text{fr}}$, $\mathcal{C}$, Cert_{cls} , and $\mathcal{F}$; specify state machine and admissible transitions
2	6–12 mo	Simulation: implement CBM/LP scenarios in SimBlock; compare GhostDAG vs PHASMDAG-Core under identical conditions
3	12–18 mo	Security analysis: tentative-order growth and certificate-issuance dynamics; quality under heterogeneous visibility; adversarial threshold as function of certification degradation
4	18–24 mo	Prototype: implement order layer + header-first propagation + dependency-aware relay + certificate issuance; expose three confirmation grades; evaluate on a controlled experimental network

3. **Incremental recomputation depth:** When $\mathcal{C}(B)$ changes (because a new descendant arrived or a predecessor was resolved), raw closure scores and certificate-side margins must be updated along the *affected descendant frontier*—not the entire DAG. The expected frontier size is bounded by the merge depth and the number of recent blocks, both of which are $O(K)$ in the current GhostDAG implementation.
4. **Lazy recomputation for deep historical blocks:** Blocks deeper than the finality depth can be considered settled; their $\mathcal{C}$ values are no longer consensus-relevant and need not be recomputed; certificates that have already become monotonic need not be re-evaluated except for audit. Pruning strategies can treat closure and certificate computation similarly to how existing GhostDAG implementations prune anticone data beyond the pruning depth.

Remark 7.1 (Computational feasibility of Approach A). Approach A is specifically designed for incremental computation: the work-weighted numerators and denominators in Equations (6) to (8) are additive or frontier-local quantities that can be updated without re-walking the entire descendant cone. This contrasts with the experimental ordering-aware extension Equation (24), which would make the same quantities part of the order-selection problem and therefore lacks a similarly clean incremental path. This is an additional reason to prefer Approach A for v_1 .

7.6 Phased Verification Roadmap

Note that Phases 1–2 can proceed in parallel with the engineering mitigations (header-first, telemetry, dependency-aware relay) and other implementation-level studies of GhostDAG. The CBM-GhostDAG security proof may be read as a decision gate: if it indicates that engineering mitigations suffice at the target BPS, PHASMDAG-Core may remain chiefly a research direction; if not, it may warrant treatment as a more substantial redesign path.

8 Discussion

8.1 Why Not Closure-Weighted Ordering in v_1 ?

Closure-weighted ordering is intellectually attractive. It appears to internalise LP/BJ risk directly into the ordering rule itself. However, it also turns closure evidence into a strategic resource. If missing closure evidence can directly reduce canonical-order attractiveness, then suppression of closure becomes a consensus-steering lever. PHASMDAG-Core therefore adopts a deliberately

Table 6: Why PHASMDAG-Core- v_1 avoids closure-weighted ordering

Dimension	Closure-weighted ordering	PHASMDAG-Core- v_1
Role of closure	direct input to ordering	certificate-gated finality layer
Effect of missing evidence	may penalise canonical order	delays Grade 2 only
Manipulation payoff	may steer consensus order	primarily latency grieving
Proof friendliness	lower	higher

conservative asymmetry: positive closure evidence unlocks stronger finality, whereas missing closure evidence does not directly punish canonical order.

The choice is therefore not that closure-weighted ordering is devoid of merit. The point is rather that it requires a separate incentive-compatibility argument, because it makes missing closure evidence a consensus-critical variable. PHASMDAG-Core- v_1 prefers semantic separation to this more aggressive design.

8.2 Relation to NC-Max

The philosophical kinship between PHASMDAG-Core and NC-Max [12] is not one of mechanism but of discipline. NC-Max introduces a chain-style two-step confirmation process so that confirmation does not outrun propagation and availability. PHASMDAG-Core imports the same discipline into the DAG setting: a robust high-throughput DAG should parallelise ordering, not finality. Strong finality is serialized through dependency closure.

In that sense, PHASMDAG-Core should not be read as an attempt to make GhostDAG’s ordering rule ever more reactive. It should be read as an attempt to prevent structural order from being mistaken for strong finality once dependency availability ceases to be well approximated by propagation completion.

8.3 Failure Modes and Design Risks

The proposal is best read alongside an explicit account of its likely failure modes. The most important of these are:

- **Certificate thresholds too conservative:** θ_{dep} , θ_{coh} , or θ_{work} may delay Grade 2 unduly, degrading usability.
- **Certificate thresholds too weak:** if the closure certificate is too permissive, strong confirmation may again outrun dependency maturity.
- **Certificate coherence slower than assumed:** if honest nodes do not converge quickly enough on certificate issuance, Grade 2 cannot be treated as a safe consensus object.
- **Frontier stability overused:** if $\mathcal{C}_{\text{fr}}(B)$ is allowed to creep back into Grade 2 semantics, local frontier steering may regain undue influence.
- **Experimental extensions misapplied:** if R_{closure} or λ are reintroduced into canonical ordering without a separate proof and incentive analysis, the protocol may recreate the very attack surface that v_1 was designed to avoid.
- **Semantic interface complexity:** the three-grade finality model may be theoretically well motivated yet difficult to expose through higher-layer interfaces without misinterpretation.

8.4 Open Problems

Certificate coherence. The central theorem target is no longer merely raw closure convergence. It is certificate coherence: whether honest nodes, given the same eventual structural evidence, converge reliably upon the same Grade 2 decisions. That is the hardest unresolved problem in the paper.

Experimental closure-weighted ordering. The earlier line of thought embodied by Equation (24) remains intellectually interesting. It may prove possible to reintroduce R_{closure} into order selection without creating unacceptable strategic leverage. But that would be a separate protocol variant, not a modest modification of PHASMDAG-Core- v_1 .

λ -bearing extensions. In PHASMDAG-Core- v_1 , λ is not consensus-critical. Any future protocol that reintroduces λ into the core rule should use only fixed consensus constants or hard-fork / epoch-finalised updates, and should prove that no divergence is introduced by parameter misalignment.

Fork distance from GhostDAG. PHASMDAG-Core should perhaps not be understood as a minor variant of GhostDAG, but rather as a more substantial protocol divergence. If ever pursued in earnest, it would probably require a distinct protocol identity (hence the auxiliary designation *RobustDAG*) rather than a simple suffix such as GhostDAG+. This bears directly upon migration, client adaptation, and ecosystem compatibility.

Three-grade confirmation interpretation cost. Moving from a single confirmation scalar to multiple semantic grades introduces an interface problem as well as a protocol problem. Any future deployment would require a careful theory of how higher layers should interpret tentative ordering, strong confirmation, and deep settlement without collapsing them back into an opaque scalar.

8.5 Protocol-Level versus Implementation-Level Mitigation

Implementation-level mitigations remain necessary, but they are not sufficient. They reduce stress; they do not close the consensus-level semantic gap by themselves. The relevant boundary is:

- **Implementation-level mitigations can reduce** propagation skew, predecessor starvation, and operator blindness.
- **Implementation-level mitigations cannot eliminate** the hidden gap between header progress and strong finality while closure fragility remains non-consensus-visible.
- **Implementation-level mitigations cannot natively certify** support structures that are individually legal but collectively closure-fragile, because that requires a protocol object such as $\text{Cert}_{\text{cls}}(B)$.
- **Implementation-level mitigations cannot provide** a protocol-native semantics for “mature enough to trust” without an object like $\text{Cert}_{\text{cls}}(B)$, grounded in raw closure observations, entering the consensus mathematics.

PHASMDAG-Core still depends on such mitigations as infrastructure:

1. **Telemetry** provides the observational basis for $\mathcal{C}(B)$ computation and certificate diagnostics (dependency completion latency and orphan-age distributions).
2. **Header-First Propagation** enables the closure layer to operate at the header level independently of body arrival, reducing the LP attack surface that the closure layer must handle.

3. **Dependency-Aware Relay** improves the convergence rate of $\mathcal{C}(B)$ by ensuring that blocks blocking the most pending successors are relayed first.

The more appropriate framing is therefore: *implementation-level mitigations are foundational, but they are not a substitute for protocol redesign; PHASMDAG-Core is the protocol-level completion that makes closure awareness explicit in the consensus mathematics.*

9 Conclusion

This paper has set out a preliminary formulation of PHASMDAG-Core, an order-first, closure-certified DAG consensus protocol motivated by the security questions raised by Wu, Wei, Zhang, and Jiang [7].

The central contention has been that the root limitation of GhostDAG-family protocols may lie not merely in the attackability of relay paths, but also in the manner in which one structural object is made to stand simultaneously for ordering, acceptability, and finality. The proposed response is to separate structural order, dependency closure, and finality into distinct mathematical roles. PHASMDAG-Core is therefore not best understood as a faster GhostDAG. It is better understood as a more semantically honest high-BPS DAG framework: tentative ordering remains fast and parallel, whereas strong finality is made conditional on closure certification.

Taken together, the present formulation supports the following tentative claims:

1. Block Jam and Late-Predecessor effects can be brought within the protocol’s mathematical core rather than treated solely as external implementation concerns.
2. A concrete and auditable v_1 instantiation of closure can be given through bounded, work-weighted support-window evidence and the monotonic certificate $\text{Cert}_{\text{cls}}(B)$.
3. The main safety benefit is semantic rather than speed-theoretic: strong confirmation is withheld until dependency closure is certified.
4. The protocol can be framed so that closure manipulation is not made to disappear, but its payoff is deliberately bounded by refusing to let it become a cheap canonical-order steering lever.
5. Semantically explicit confirmation grades can be coupled to explicit invariants rather than collapsed into a single opaque scalar.

The principal unresolved theorem target is certificate coherence: whether honest nodes, once presented with sufficiently mature structural evidence, converge reliably upon the same Grade 2 decisions. Approach A has been standardised as the reference v_1 design, while closure-weighted ordering remains future work. The paper has therefore offered not a completed proof, but a conservative protocol proposal together with a phased programme of formal modelling, simulation, and implementation.

Whether PHASMDAG-Core ultimately warrants treatment as an independent protocol family—or, in the auxiliary language used here, as a genuine RobustDAG design—must depend upon the outcome of that programme.

Acknowledgements

The author thanks colleagues and interlocutors whose discussions on DAG consensus, delayed acceptability, and finality semantics helped sharpen the questions addressed in this paper. Any remaining errors, omissions, or overstatements are the sole responsibility of the author.

References

- [1] Y. Sompolinsky and A. Zohar, “Secure High-Rate Transaction Processing in Bitcoin,” *IACR Cryptology ePrint Archive*, Report 2013/881, 2013.
- [2] J. Garay, A. Kiayias, and N. Leonardos, “The Bitcoin Backbone Protocol: Analysis and Applications,” *Advances in Cryptology – EUROCRYPT*, 2015.
- [3] I. Eyal and E. G. Sirer, “Majority Is Not Enough: Bitcoin Mining Is Vulnerable,” *arXiv preprint arXiv:1311.0243*, 2013.
- [4] Y. Sompolinsky and A. Zohar, “SPECTRE: A Fast and Scalable Cryptocurrency Protocol,” *IACR Cryptology ePrint Archive*, Report 2016/1159, 2016.
- [5] Y. Sompolinsky and A. Zohar, “PHANTOM: A Scalable BlockDAG Protocol,” *IACR Cryptology ePrint Archive*, Report 2018/104, 2018.
- [6] Y. Sompolinsky, S. Wyborski, and A. Zohar, “PHANTOM GHOSTDAG: A Scalable Generalization of Nakamoto Consensus,” *Proceedings of the 3rd ACM Conference on Advances in Financial Technologies (AFT)*, pp. 57–70, 2021.
- [7] S. Wu, P. Wei, R. Zhang, and B. Jiang, “Security-Performance Tradeoff in DAG-based Proof-of-Work Blockchain Protocols,” *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2024.
- [8] R. Zhang and B. Preneel, “Lay Down the Common Metrics: Evaluating Proof-of-Work Consensus Protocols’ Security,” *Proceedings of the 40th IEEE Symposium on Security and Privacy (S&P)*, pp. 1441–1458, 2019.
- [9] H. Yu, “OHIE: Blockchain Scaling Made Simple,” *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, pp. 908–925, 2021.
- [10] V. Bagaria, S. Kannan, D. Tse, G. Fanti, and P. Viswanath, “Prism: Deconstructing the Blockchain to Approach Physical Limits,” *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pp. 585–602, 2019.
- [11] Y. Aoki, K. Shigeta, and R. Banno, “SimBlock: A Validated Blockchain Simulator,” *Proceedings of the IEEE International Conference on Blockchain (Blockchain)*, pp. 131–139, 2019.
- [12] L. Zhang, F. Zhang, and J. Sun, “NC-Max: Breaking the Security-Performance Tradeoff in Nakamoto Consensus,” *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2022.

Algorithm 1 PHASMDAG-Core Block Processing

Require: Incoming block B_{new} with parent set P

Ensure: Updated DAG state Σ

```
1: // — Layer A: Order —
2: Validate header in isolation (PoW, timestamp, format)           ▷ Same as GhostDAG
3: Check all parents exist in  $\Sigma_{\text{ord}}$                              ▷ check_parents_exist
4: Compute  $\mathcal{O}(B_{\text{new}})$  and  $r(B_{\text{new}})$  via blue-work ordering
5: Select preferred parent, compute mergeset, colour blue/red
6: Update  $\Sigma_{\text{ord}}(B_{\text{new}})$ 

7: // — Layer B: Closure —
8: Identify affected ancestors  $A$  whose support windows include  $B_{\text{new}}$ 
9: for each affected block  $A$  do
10:   Update  $\mathcal{W}_H(A)$  and  $\mathcal{U}(A)$ 
11:   Recompute  $\mathcal{C}_{\text{dep}}(A)$ ,  $\mathcal{C}_{\text{coh}}(A)$ , and  $\mathcal{C}_{\text{fr}}(A)$            ▷ Approach A /  $v_1$ 
12:    $\mathcal{C}(A) \leftarrow g_{v_1}(\mathcal{C}_{\text{dep}}(A), \mathcal{C}_{\text{coh}}(A), \mathcal{C}_{\text{fr}}(A))$ 
13:   Compute  $\text{Work}_H(A)$ 
14:   Recompute  $\mathcal{C}_{\text{dep}}^{\text{cert}}(A)$  and  $\mathcal{C}_{\text{coh}}^{\text{cert}}(A)$ 
15:   if  $\mathcal{C}_{\text{dep}}^{\text{cert}}(A) \geq \theta_{\text{dep}} \wedge \mathcal{C}_{\text{coh}}^{\text{cert}}(A) \geq \theta_{\text{coh}} \wedge \text{Work}_H(A) \geq \theta_{\text{work}}$  then
16:     Set  $\text{Cert}_{\text{cls}}(A) \leftarrow 1$                                ▷ Monotonic once issued
17:   end if
18: end for
19: Initialise  $\Sigma_{\text{cls}}(B_{\text{new}})$ 

20: // — Layer C: Finality —
21: Compute Grade predicates using Equations (15) to (17)
22: Assign confirmation grade:
23: if  $\text{Grade}_3(B)$  then
24:   Grade 3: Deep Settlement
25: else if  $\text{Grade}_2(B)$  then
26:   Grade 2: Strong Confirmation
27: else if  $\text{Grade}_1(B)$  then
28:   Grade 1: Tentative Ordering
29: else
30:   No confirmation
31: end if
32: Update  $\Sigma_{\text{fin}}(B_{\text{new}})$  and affected blocks
```

A Relationship to GhostDAG’s K Parameter

GhostDAG’s security parameter K is derived from the network delay bound D and block production rate f under UDBM assumptions [6]:

$$K = f(2Df, \delta) \quad (25)$$

where δ is the tail probability that the anticone of a block exceeds K . The derivation assumes that blocks are accepted within delay D .

Under CBM, the effective delay is $\Delta_{\text{actual}} = D + \Delta_{\text{proc}}$, which may exceed D . A naïve adaptation would replace D with $D_{\text{eff}} = D + \mathbb{E}[\Delta_{\text{proc}}]$:

$$K_{\text{naive}} = f(2D_{\text{eff}}f, \delta) \quad (26)$$

However, this approach has two problems:

1. $\mathbb{E}[\Delta_{\text{proc}}]$ is not a fixed constant but depends on network conditions and attack intensity, making K_{naive} unstable.
2. Changing K is a hard consensus change—all nodes must agree on the same K —but D_{eff} varies across nodes under LP attack.

PHASMDAG-Core resolves this differently. The closure layer does *not* alter the consensus value of K , nor does it reweight an otherwise valid ordered block inside the order core. Instead, effective delay exposure appears in the transition from tentative order to strong confirmation:

- The K parameter remains a consensus constant derived from the optimistic D (as in current GhostDAG).
- Tentative order is still computed from the GhostDAG order core under that fixed K .
- The closure layer determines whether an already ordered block may advance from Grade 1 to Grade 2 by obtaining $\text{Cert}_{\text{cls}}(B) = 1$.
- Under LP attack, the observable degradation therefore appears as a lower certificate-issuance rate $p_{\text{cert}}(T)$ rather than as a consensus-visible increase in K .

This design choice is deliberate: it keeps the consensus rule simple and stable (K is fixed) while allowing adaptive delay awareness to enter through certificate-gated finality. The tradeoff is that strong-finality growth may degrade under LP attack even while tentative order continues to grow according to the underlying GhostDAG order core.

Table 7: Parameter handling comparison

Approach	Consensus K	Delay handling
GhostDAG (UDBM)	$K = f(2Df, \delta)$	D assumed uniform
GhostDAG + adaptive K	$K = f(2D_{\text{eff}}f, \delta)$	Consensus risk: nodes disagree on D_{eff}
PHASMDAG-Core	$K = f(2Df, \delta)$ (unchanged)	Order unchanged; delay appears in certificate issuance $\text{Cert}_{\text{cls}}(B)$